TECHNICAL AND PLAIN-LANGUAGE COMPANION

Explanation Week 6

Immutable API/Next pairs, health-gated Nginx cutover, and exact rollback.

This file explains

- Why API and Next are promoted as one content-addressed release pair.
- How the idle color is validated before Nginx changes stable traffic.
- Why continuous transition and target-only observation gates must pass before state is committed.
- How rollback restores the exact prior application pair without reversing shared data or migrations.

FLOW

Manifest -> idle pair -> health + identity -> Nginx switch -> observation -> state commit

KEY GUARANTEE

Promotion and rollback fail closed. The former pair stays warm, while SQL, queues, workers, Vault, and volumes remain shared.

Source baseline: current workspace, 25 August 2026.

1. Paired promotion code

Source: `infra/week6/deploy.sh:76-146` (condensed without changing order)

```
76  inspect and validate the release manifest
87  inspect current deployment state
91  choose the opposite idle color; first deployment chooses blue
97  write exact manifest images into release.env
98  validate the composed configuration
101 ensure exact API image exists
102 ensure exact Next image exists
103 ensure exact migrator image exists
105 run the compatible one-shot migrator
108 start idle API; wait healthy
110 start idle Next; wait healthy
113 create timestamped evidence path
115 run idle smoke; verify color and revision
117 preserve the active Nginx include as prior
122 write and validate candidate upstream include
127 start continuous former-to-target cutover probe
131 mark traffic switch in progress
132 atomically switch Nginx to candidate include
133 require the cutover probe to pass
141 run target-only observation probe
146 commit release state only after every gate passes
```

Stage	Why it exists
`Manifest`	API and Next are one immutable release unit; no target-host rebuild.
`Migrator`	Forward-compatible schema change runs once before new application traffic.
`Idle health`	A container is not promotable merely because it started; both processes must be healthy and identify correctly.
`Nginx validation`	Catch an invalid route before the active include changes.
`Continuous probe`	Prove requests survive the transition and observe only allowed release identities.
`Observation`	Prove stable traffic settles entirely on the target.
`State commit`	Only a verified cutover becomes the new rollback reference.

> **Plain language:** stock and inspect the closed storefront, test its doors, validate the sign mechanism, watch customers while flipping the sign, then write the official change in the release ledger.

Failure cleanup

If a post-switch gate fails before commit, the script restores the previous include and stops the uncommitted candidate. Evidence and candidate logs remain for diagnosis.

2. Rollback code and limits

Source: `infra/week6/rollback.sh:62-122` (condensed)

```
62  read current and previous release state
68  fail if no verified prior release exists
70  select the archived previous manifest (or exact supplied file)
76  fail if the manifest is unavailable
84  validate exact hash/revision and write rollback environment
89  ensure exact prior API image
90  ensure exact prior Next image
92  announce that no migration will run
93  start prior API; wait healthy
95  start prior Next; wait healthy
99  run idle smoke against the exact prior revision
102 preserve current active include
103 write and validate prior pair as candidate
105 run continuous rollback cutover probe
110 atomically switch Nginx
111 require transition probe success
114 run target-only observation
119 commit rollback state
122 report no migration or shared-state reversal
```

Why rollback does not reverse the database

Both colors share SQL. Automatically undoing schema while another process uses it is unsafe. Week 6 requires compatible forward migrations: the prior application must remain able to run during the rollback window.

Question	Answer
Does rollback rebuild?	No. It restores exact content-addressed images from the verified prior manifest.
Does it migrate?	No migration and no down migration.
Does it lose queued work?	No. SQL, Outbox, RabbitMQ, workers, Inbox, Vault, and volumes are shared and remain running.
Why keep the former color?	Warm fallback makes rollback fast and independently health-checkable.
When can it be retired?	Only after an explicit rollback-window decision; retirement is not part of the normal demo.

> **Storefront analogy:** rollback points the sign to the prior inspected storefront. It does not rewind the shared stockroom, so stockroom changes must remain compatible with both storefronts during the rollback window.

Review summary

Week 6 boundaries, evidence, and short answers

Failure behavior

- Candidate health or identity failure causes no cutover; logs and evidence remain for diagnosis.
- A post-switch gate failure before commit restores the prior Nginx include automatically.
- Rollback fails if it cannot prove the archived prior manifest and exact content-addressed images.
- Rollback runs no migration and reverses no SQL, RabbitMQ, Vault, or other shared state.

Review answers

- The API and Next frontend form one release unit because clients must observe a compatible pair.
- Forward-compatible migrations keep both colors usable during the rollback window.
- The former color remains warm so rollback can be health-checked and fast.
- Workers have no colors because they continuously drain the shared durable workflow.

Executable evidence

- Week 6 transition and rollback evidence record the manifest, identities, probes, cutover, and final active pair.

The detailed pages preserve the original source excerpts; headings and footers are renumbered for this standalone file.